1.2.2. Using concurrency for performance

Multiprocessor systems have existed for decades, but until recently they were mostly found only in supercomputers, mainframes, and large server systems. But chip manufacturers have increasingly been favoring multicore designs with 2, 4, 16, or more processors on a single chip over better performance with a single core. Consequently, multicore desktop computers, and even multicore embedded devices, are now increasingly prevalent. The increased computing power of these machines comes not from running a single task faster but from running multiple tasks in parallel. In the past, programmers have been able to sit back and watch their programs get faster with each new generation of processors, without any effort on their part. But now, as Herb Sutter put it, "The free lunch is over."[1] *If software is to take advantage of this increased computing power, it must be designed to run multiple tasks concurrently*. Programmers must therefore take heed, and those who have hitherto ignored concurrency must now look to add it to their toolbox.

[1] "The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software," Herb Sutter, *Dr. Dobb's Journal*, 30(3), March 2005. http://www.gotw.ca/publications/concurrency-ddj.htm.

There are two ways to use concurrency for performance. The first, and most obvious, is to divide a single task into parts and run each in parallel, thus reducing the total runtime. This is *task parallelism*. Although this sounds straightforward, it can be quite a complex process, because there may be many dependencies between the various parts. The divisions may be either in terms of processing—one thread performs one part of the algorithm while another thread performs a different part—or in terms of data—each thread performs the same operation on different parts of the data. This latter approach is called *data parallelism*.

Algorithms that are readily susceptible to such parallelism are frequently called *embarrassingly parallel*. Despite the implications that you might be embarrassed to have code so easy to parallelize, this is a good thing: other terms I've encountered for such algorithms are *naturally parallel* and *conveniently concurrent*. Embarrassingly parallel algorithms have good scalability properties—as the number of available hardware threads goes